Keyostar - Design Document

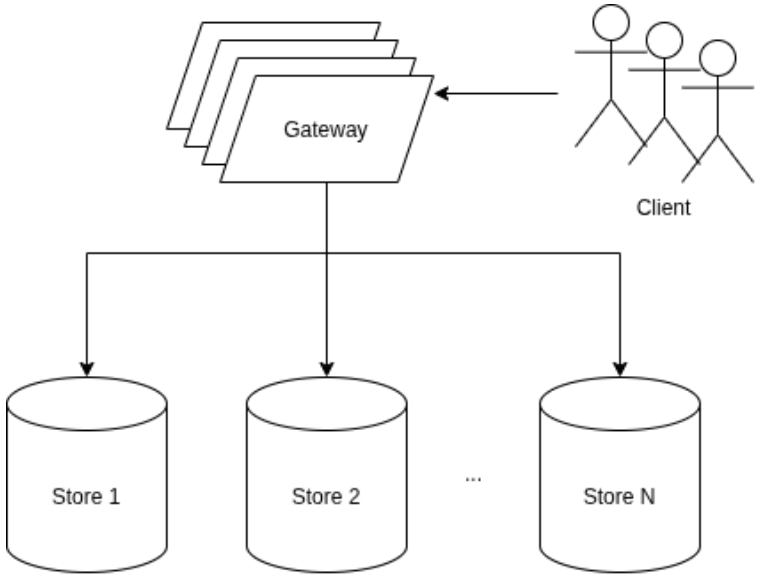
Keyostar is a simple and efficient distributed in-memory key-value store implemented using Java and Spring Boot.

This document provides an overview of the system architecture, the tradeoffs encountered during its design, the alternatives considered, and the reasoning behind the resulting design choices.

1. System Architecture

1.1 Overview

Keyostar consists of two main components: the **gateway** and the **stores**.



The data is horizontally partitioned across multiple shards, which are referred to throughout this document as **stores**. The number of store instances is static and configured before deployment. This value cannot be changed at runtime; in other words, dynamic scaling of the store layer is not supported.

Determining which store is responsible for a given key is the responsibility of the **gateway**. Any number of gateway replicas may exist, and the number of gateway instances can be increased or decreased at runtime, for example for load-balancing purposes. Gateway instances are stateless and interchangeable.

All client requests are intended to pass through a gateway instance. Stores are not intended to receive requests directly from clients, although this is not explicitly prevented in the current implementation and can be useful for testing.

For GET, PUT, and DELETE requests, the gateway determines the target store from the key and forwards the request to that store. If the selected store is unreachable or does not respond, the operation fails and the gateway returns an error response to the client.

1.2 Store Layer

Each store instance is responsible only for the portion of the data assigned to it. A store has no knowledge of the existence of other stores or of the gateway.

Data is stored in memory using a ConcurrentHashMap and is not persisted to disk. Consequently, all data held by a store is lost if that instance is shut down or restarted.

A store supports safe concurrent access to its underlying hash map. Individual operations such as get, put, and delete are atomic, allowing multiple clients to access the same store instance concurrently without corrupting its state.

Compound operations are intentionally not implemented. For example, an operation that reads multiple records and derives a new record from them would not automatically be atomic and would require additional synchronization mechanisms.

1.3 Replication and Availability

The stores are not replicated. Each store instance therefore represents the single source of truth for the portion of the key space assigned to it.

If a store instance becomes unavailable, the data assigned to that store also becomes unavailable. If the instance is permanently lost, its data cannot be recovered.

This design choice was made primarily for simplicity and scope. Introducing replication would significantly increase the complexity of the system. A replicated design would require, among other things, mechanisms for replica health detection, leader or coordinator election, update propagation, synchronization between replicas, consistency guarantees, and recovery behavior.

For this reason, Keyostar favors a simple fail-fast behavior: if the store responsible for a key is unavailable, operations targeting that key fail rather than being redirected to another replica or served from a potentially stale state.

2. Hashing, Partitioning, and Addressing

To distribute data evenly across store instances, keys should be mapped as uniformly as possible. Since Keyostar is a general-purpose key-value store, there is

no a priori knowledge of the distribution of keys or their access patterns, e.g. whether keys are timestamps and recent values are accessed more frequently.

For this reason, Keyostar uses a general-purpose hash function. For simplicity, the current implementation uses Java's built-in string hashing method.

The hash function can be replaced by an alternative such as MurmurHash if better distribution characteristics or hashing performance are desired. However, the expected impact on end-to-end request latency is likely to be small, since gateway processing time is dominated by HTTP/network communication and framework overhead rather than the hash computation itself.

The DefaultPartitioner uses the configured hash function together with Math.floorMod to determine the target store index:

$$storeIndex = hash(key) \bmod \#stores$$

Unlike the % operator, Math.floorMod produces a non-negative result when the divisor is positive, which is important because Java hash codes may be negative. If the divisor is zero, an ArithmeticException is thrown. Therefore, the configured store count must always be at least 1.

Once the partitioner determines the store index, the AddressResolver resolves that index to the URI of the corresponding store.

Three addressing strategies are supported:

- localhost
- docker
- kubernetes

Each strategy reflects the addressing scheme of its respective deployment environment while keeping the partitioning logic independent of deployment-specific details.